Лабораторная работа №3: Создание базы данных в СУБД PostgreSQL

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ 3

УСЛОВИЯ ДЛЯ НАЧАЛА РАБОТЫ 5

СОСТАВ ОТЧЁТА 6

ЗАДАНИЯ 7

ВСПОМОГАТЕЛЬНЫЕ МАТЕРИАЛЫ 10

1 НАЧАЛО РАБОТЫ 11

1.1 Создание БД в pgAdmin IV 11

1.2 Создание БД в dbeaver 14

1.3 Создание БД в psql 17

1.4 Язык SQL 18

2 СОЗДАНИЕ ТАБЛИЦ 20

2.1 Ограничения 21

2.2 Создание таблицы «Секция» 26

2.3 Создание таблицы «Типы товаров» 30

2.4 Создание таблицы «Инвентарь» 31

2.5 Создание таблицы «Состав секций» 33

2.6 Создание таблицы «Услуга» 34

2.7 Создание таблицы «Сотрудники» 35

2.8 Создание таблицы «Товар» 39

2.9 Создание таблицы «Заказы» 40

3 ИЗМЕНЕНИЕ ТАБЛИЦ 42

3.1 Переименование таблицы 42

3.2 Добавление столбцов 42

3.3 Удаление столбцов 44

3.4 Переименование столбцов 44

3.5 Изменение типов данных столбцов 45

3.5.1 Расширение и сужение столбцов 45

3.5.2 Изменение типов данных 46

3.6 Работа с ограничениями 47

3.6.1 Изменение обязательности 48

3.6.2 Добавление первичного ключа 49
3.6.3 Добавление внешнего ключа 49
3.6.4 Работа с другими ограничениями 49
4 УДАЛЕНИЕ ТАБЛИЦ 51
5 ЗАПОЛНЕНИЕ ТАБЛИЦ 52
6 ОБНОВЛЕНИЕ ДАННЫХ В ТАБЛИЦЕ 57
7 УДАЛЕНИЕ ДАННЫХ ИЗ ТАБЛИЦ 59

ВВЕДЕНИЕ

Данная лабораторная работа посвящена созданию базы данных в СУБД PostgreSQL. Выполнив её, вы научитесь создавать таблицы, менять их структуру, связывать таблицы друг с другом, заполнять таблицы данными, удалять и очищать таблицы, работать с ограничениями и так далее.

УСЛОВИЯ ДЛЯ НАЧАЛА РАБОТЫ

Для начала работы над лабораторной у вас в обязательном порядке должно быть завершено проектирование базы данных, что является основой для выполнения данной работы. Без правильной логической и физической модели выполнять эту работу не имеет смысла.

СОСТАВ ОТЧЁТА

В отчёте необходимо указать, в каком клиенте велась работа (pgAdmin/psql/dbeaver). Указать каждый запрос, который выполнялся в рамках лабораторной работы. Обязательно наличие описания шагов.
ЛУЧШЕ ВСЁ-ТАКИ ИСПОЛЬЗОВАТЬ DBEABER!

ЗАДАНИЯ

1. Установить и настроить PostgreSQL (желательно использовать версию сервера PostgreSQL >= 10) и определиться с клиентом для работы (pgAdmin и psql устанавливаются вместе с PostgreSQL, dbeaver - отдельно).

В отчёте не забыть указать версию сервера, название и версию клиента.

2. Создать базу данных в выбранном клиенте (pgAdmin, dbeaver или в консоли psql).

!Сделать скриншоты процесса создания БД.!

3. На основании логической и физической модели создать каждую таблицу (запрещено создание таблиц через визуальный интерфейс - создавать только запросами вручную, нельзя также создавать таблицы через интерфейс и копировать сгенерированный запрос). Связать необходимые таблицы между собой. Реализовать ограничения, если они есть. Указать, какие ограничения не будут реализовываться на уровне базы данных

!Сохранить текстовые версии запросов.!

4. Научиться изменять структуру таблиц: изменять уже созданные столбцы, удалять столбцы, добавлять столбцы; добавлять, изменять и удалять ограничения; переименовывать таблицы, ограничения и т.д.

!Сохранить текстовые версии запросов.!

4.1. Переименование таблиц (переименовать любую таблицу, а затем вернуть ей прежнее имя);

4.2. Создать тестовую таблицу с 1-2 любыми столбцами. Не указывать первичный ключ

4.2.1. Добавить 1 обычный столбец в эту таблицу;

4.2.2. Добавить 1 столбец - первичный ключ в таблицу.

4.3. Удаление столбцов из таблиц (удалить добавленные ранее столбцы из таблицы);

4.4. Переименование столбцов (переименовать любой столбец, а затем вернуть ему прежнее имя);

4.5. Изменение типа данных столбцов (изменить типы данных нескольких столбцов (число в строку, строку в число, дату в строку, строку в дату)).

4.6. Изменение ограничений столбцов. Создать 2 тестовых таблицы, которые должны быть связаны друг с другом (изначально не связывать их). Добавить в одну из таких таблиц ограничения PRIMARY KEY, FOREIGN KEY, CHECK, DEFAULT, UNIQUE, а также каскадное удаление записей.

5.Удаление таблиц. Удалить все тестовые таблицы, созданные ранее.

6.Заполнить КАЖДУЮ ТАБЛИЦУ данными с помощью INSERT-запросов.

!Сохранить текстовые версии запросов.!

Для выполнения следующих лабораторных работ необходимо некоторое минимальное количество данных в некоторых таблицах. Поэтому рекомендуется заполнить ваши БД хотя бы указанными данными. Данные должны отличаться. То есть если в варианте есть продажа товаров, то должны быть отличающиеся друг от друга заказы (например, по количеству товаров в заказе). Если это сотрудники организации, то следует хотя бы кому-то дать другую должность (если должности есть). Это как пример.

Вариант	Минимальное количество данных
Учёт книг в библиотеке	9-10 разных книг (у некоторых одинаковый жанр). 5-6 выдач и возвратов. 1-2 долга.
Сеть книжных магазинов	9-10 разных книг (у некоторых одинаковый жанр). 7-8 продаж с разным составом товаров.
Вакансии	5-6 вакансий на разные должности. 1-3 соискателя на каждую должность (но на 1-2 должности нужно хотя бы 2 соискателя).
Отдел кадров (внутри компании)	Несколько подразделений с 3-4 сотрудниками в каждом.
Студенты	3-4 группы с 5-6 студентами в каждой. Несколько предметов.

	Заполненная успеваемость для каждого студента. Кого-нибудь сделать должником.
Учёт товаров на складе	Несколько единиц оборудования. 9-10 товаров разных категорий (у некоторых категории одинаковые). Несколько поставок и отгрузок товара разного объёма.
Перевозки	Несколько маршрутов с расписанием и несколько автобусов. По несколько проданных билетов на каждый рейс. Также сделать так, чтобы было несколько проданных билетов на пару-тройку разных рейсов по одному маршруту.
Вычислительная техника	4-5 различных видов техники, состоящих из разного числа материалов.
Продажа автомобилей	5-6 разных машин (можно одной марки). 2-3 проданных автомобиля.
Домашняя библиотека	9-10 разных книг разных жанров. 2-3 книги сделать с одним жанром. Часть книг сделать не своими, а взятыми где-нибудь.
Программы	5-6 программ разных типов.
Кинотеатр	3-4 фильма с разным числом сеансов в разное время. 5-6 купленных билетов на каждый сеанс. Сделать так, чтобы было хотя бы 2 разных сеанса на один фильм в разное время.
Перевозки	3-4 машины. 7-8 доставок. Несколько доставок должно быть в разные места.
Классный руководитель	7-8 учеников в одном классе. 3-4 предмета. Заполненная успеваемость.
Оптовая база	9-10 товаров разных категорий. 3-4 поставки. 3-4 отгрузки. Разный объём поставок и отгрузок.

Детский сад	2-3 группы с 5-6 детьми в каждой.
Рецепты	4-5 блюд с заполненными ингредиентами и рецептами.
Прокат автомобилей	6-7 автомобилей. 3-4 проката.
Химчистка	3-4 услуги. 5-6 оказанных услуг клиентам.
Преподавателей	5-6 преподавателей. 5-6 дисциплин. Заполненное расписание.

7.Обновить 2-3 записи в таблицах.

8.Удалить 1-2 записи в таблицах.

ВСПОМОГАТЕЛЬНЫЕ МАТЕРИАЛЫ

В этом разделе представлены материалы, которые использовались для создания этой лабораторной работы и которые помогут при её выполнении.

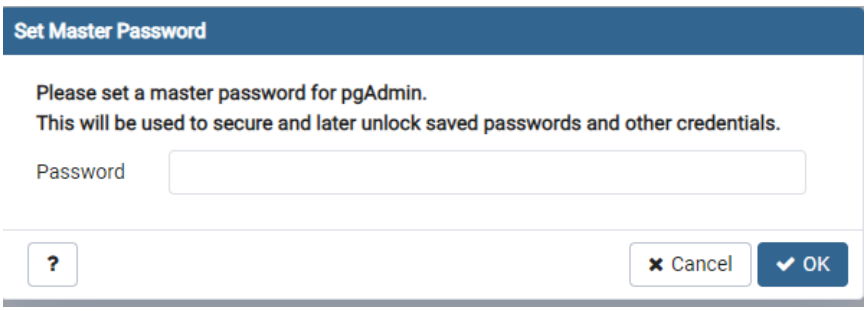
Материал	Источник
Документация PostgreSQL (eng)	https://www.postgresql.org/docs/15/index.html
Документация PostgreSQL (ru)	https://postgrespro.ru/docs/postgresql/15/index

1 НАЧАЛО РАБОТЫ

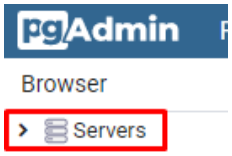
Далее вся работа будет представлена сразу в трёх разных клиентах. Работать нужно только в одном из них.

1.1 Создание БД в pgAdmin IV

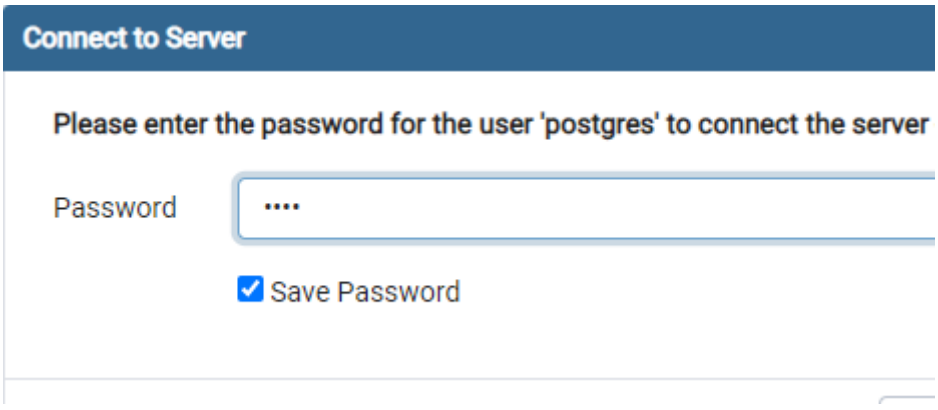
Запуск pgAdmin IV. При первом запуске нужно будет ввести мастер-пароль. Его нужно будет вводить каждый раз при запуске pgAdmin IV.



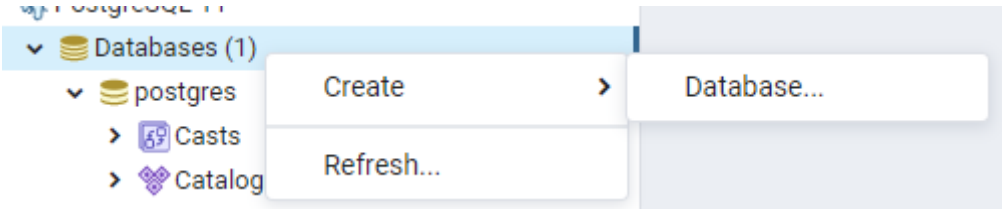
Нужно раскрыть список серверов.

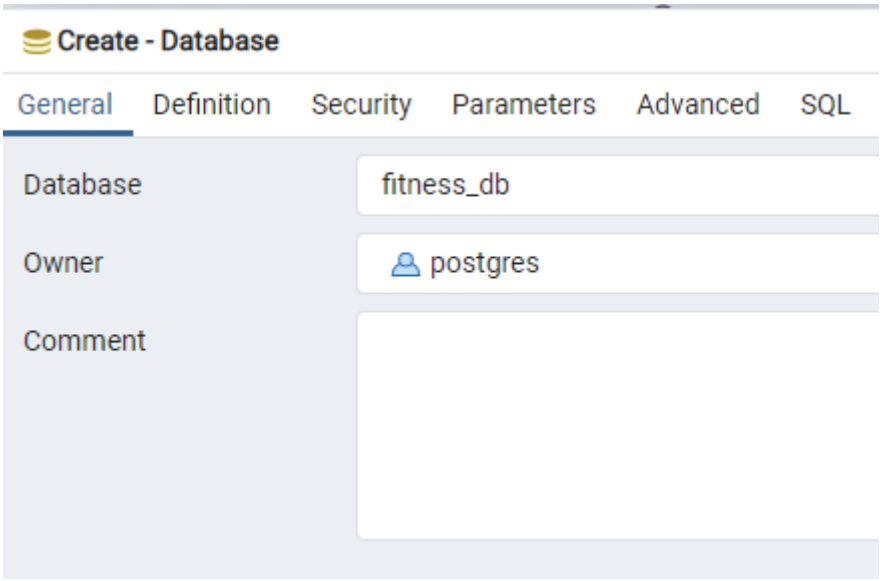


На этом этапе pgAdmin попросит ввести пароль от пользователя-владельца БД. Этот пароль указывался при установке PostgreSQL. Можно нажать «Save Password», чтобы не вводить этот пароль при каждом открытии pgAdmin.

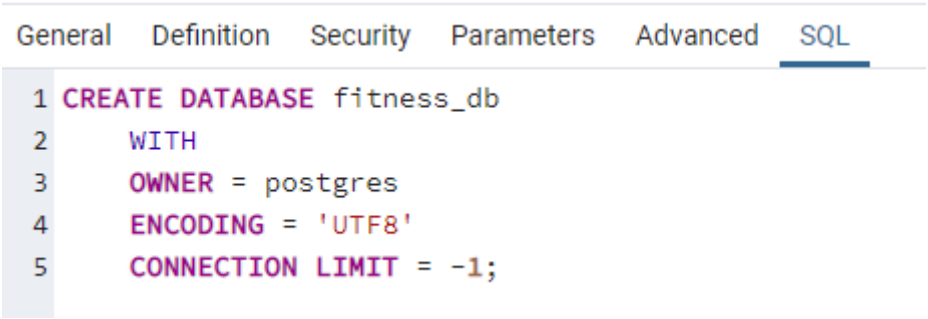


Далее нужно создать свою базу данных.

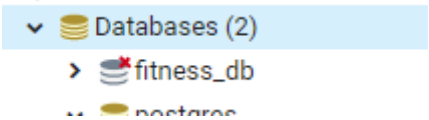




На вкладке SQL можно будет посмотреть сгенерированный код для создания базы данных.



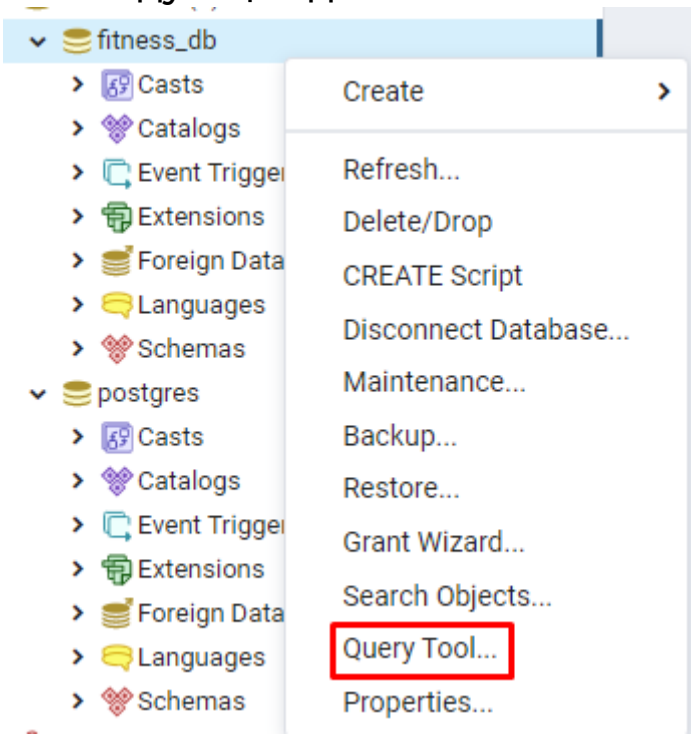
Затем нужно подключиться к созданной БД. Просто выделить её.



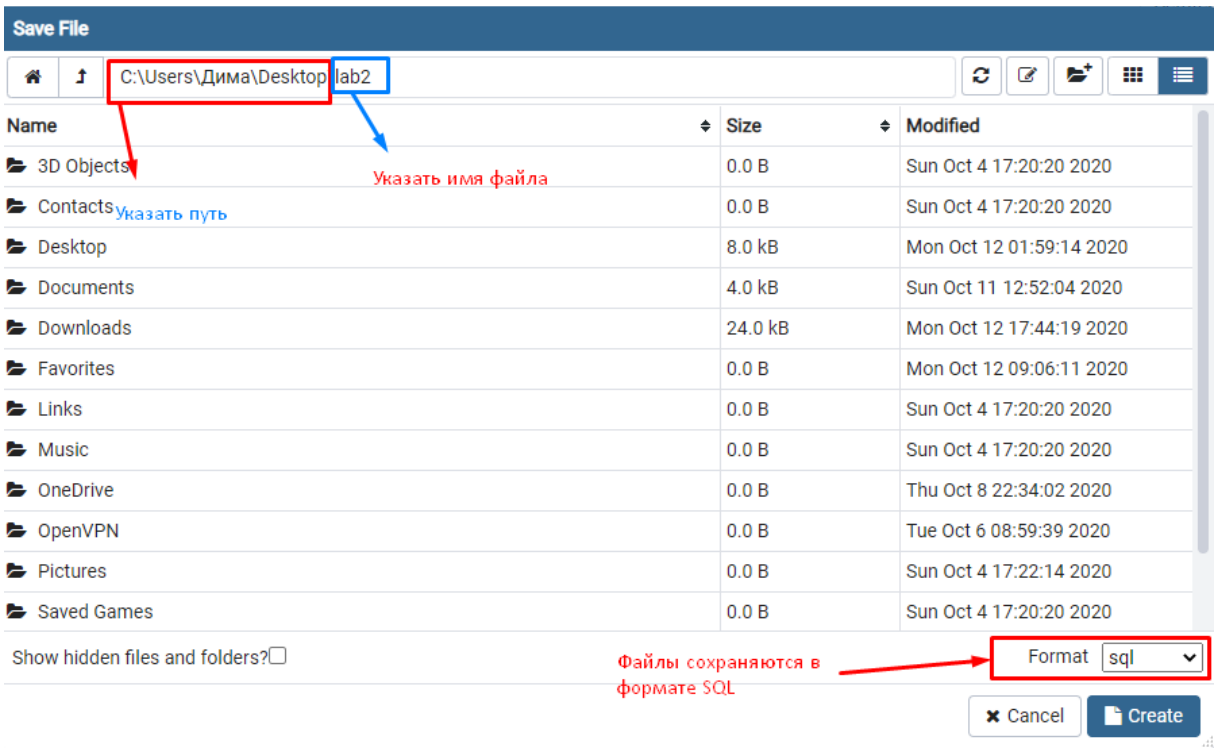
После этого можно начинать создавать таблицы. В pgAdmin это можно сделать без написания кода. Но в лабораторной так делать запрещено. Создавать таблицы нужно самостоятельно с помощью написания CREATE-запросов. Также запрещено создавать таблицы через интерфейс и выдавать

сгенерированный СУБД код в качестве своего. В этом случае работа не считается выполненной.

Код пишется в редакторе запросов. Чтобы его открыть, нужно выполнить следующее действие:

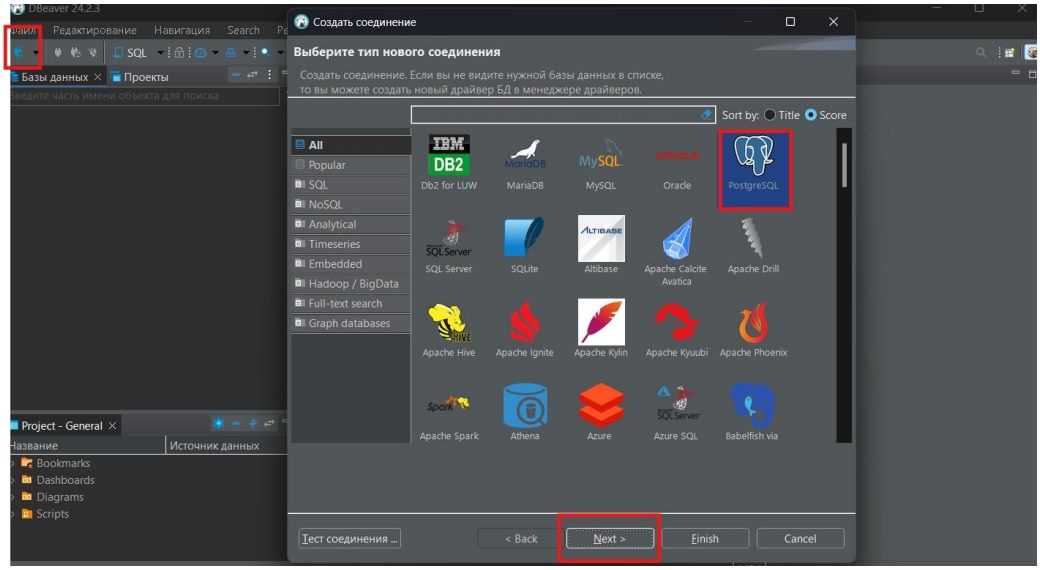


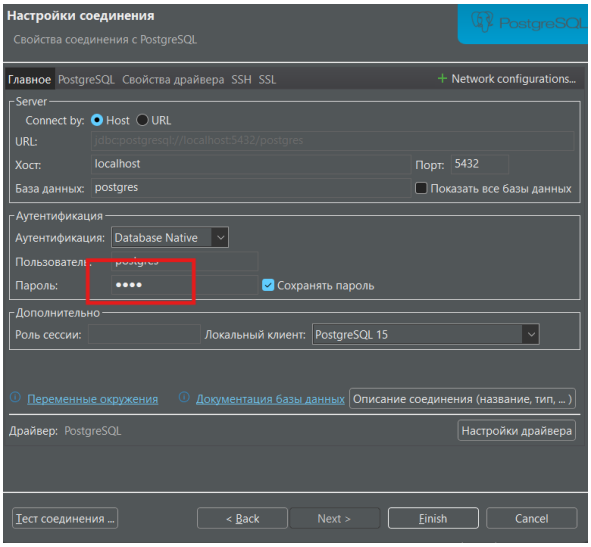
Процесс написания кода будет более подробно рассмотрен далее. А пока очередное наставление - не забывайте сохранять свой код именно в текстовом формате.



1.2 Создание БД в dbeaver

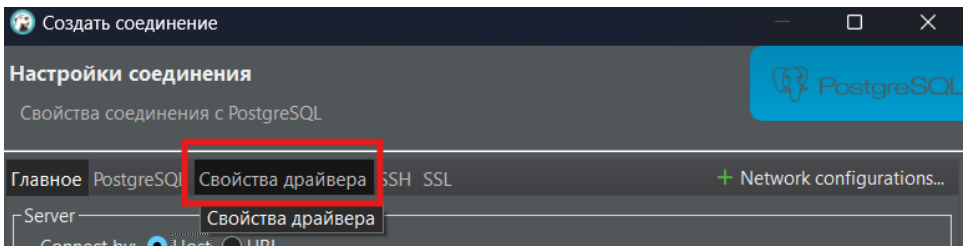
Нужно нажать на кнопку «Новое соединение».
Выбрать postgresql.

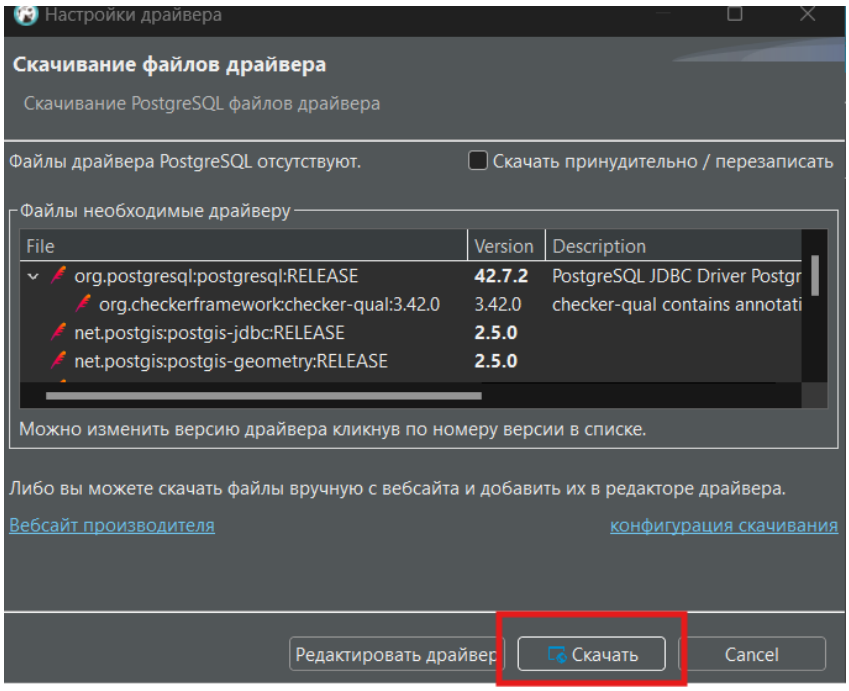




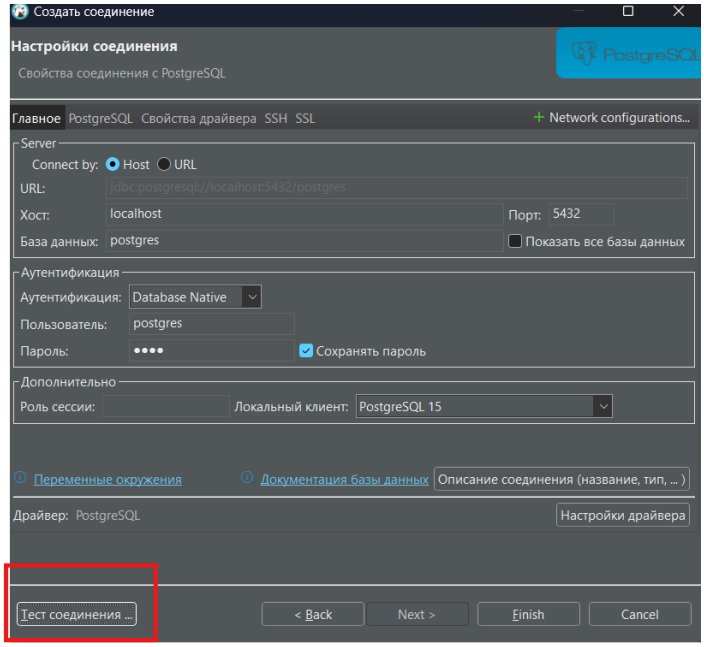
Вводим пароль от стандартного пользователя. Этот пароль вы задавали при установке PostgreSQL.

Далее необходимо скачать драйверы

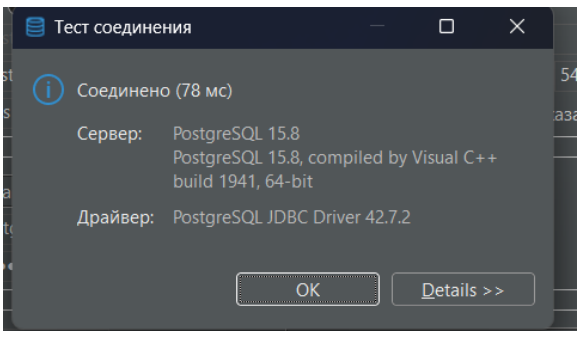




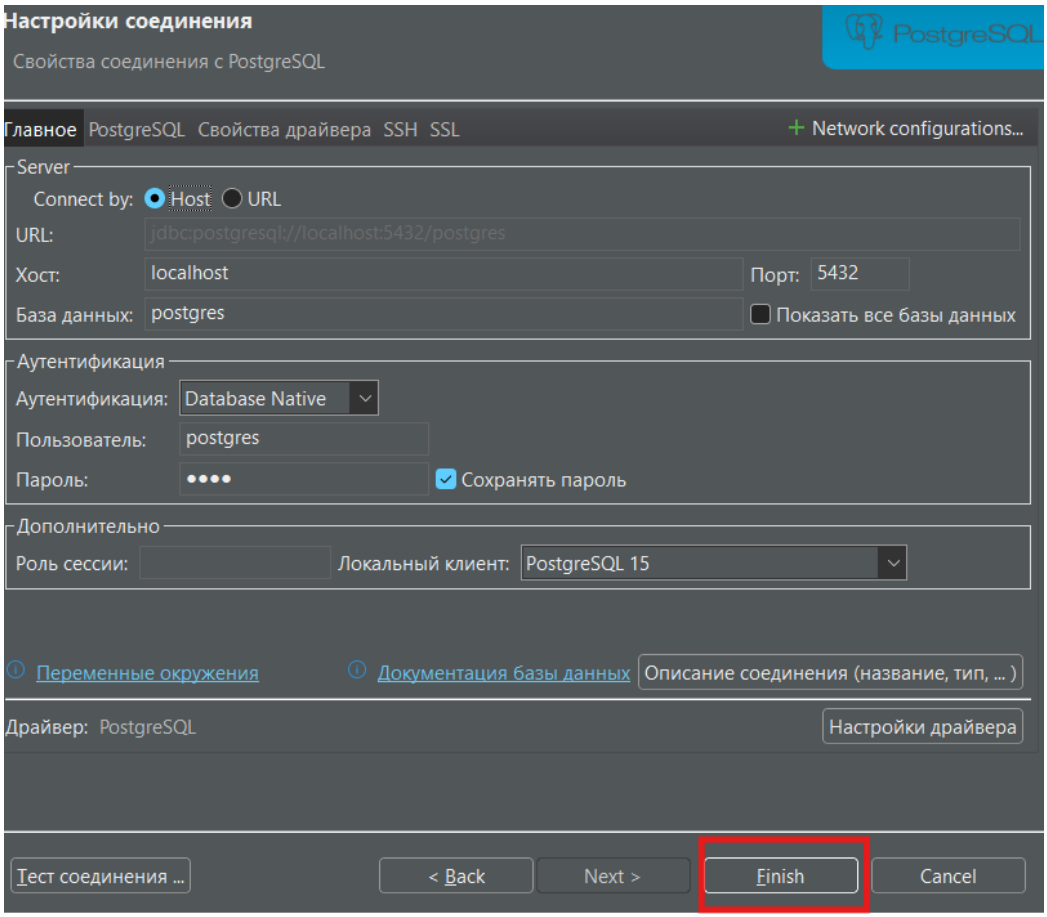
Далее возвращаемся во вкладку «Главное» и нажимаем «Тест соединения».



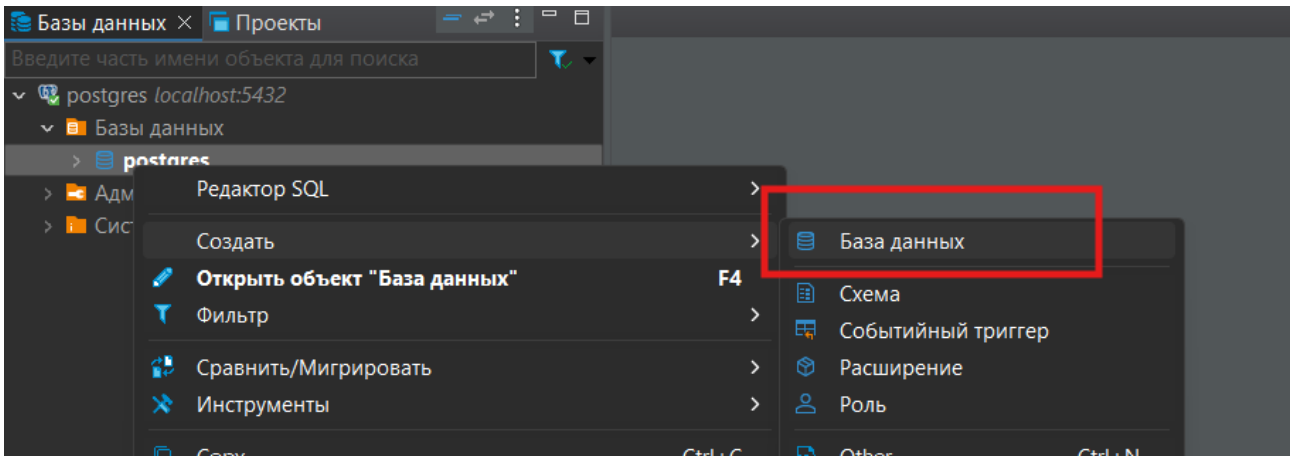
Если все сделано правильно выйдет такое окно:

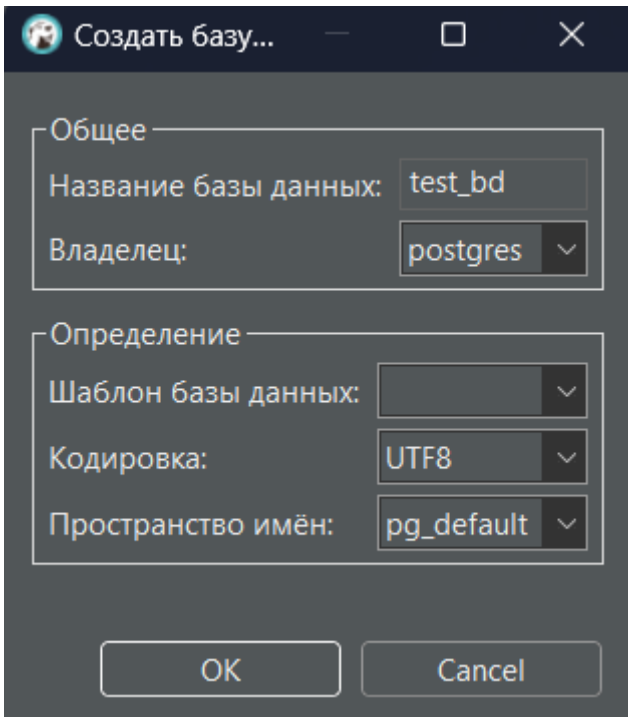


Заканчиваем настройку

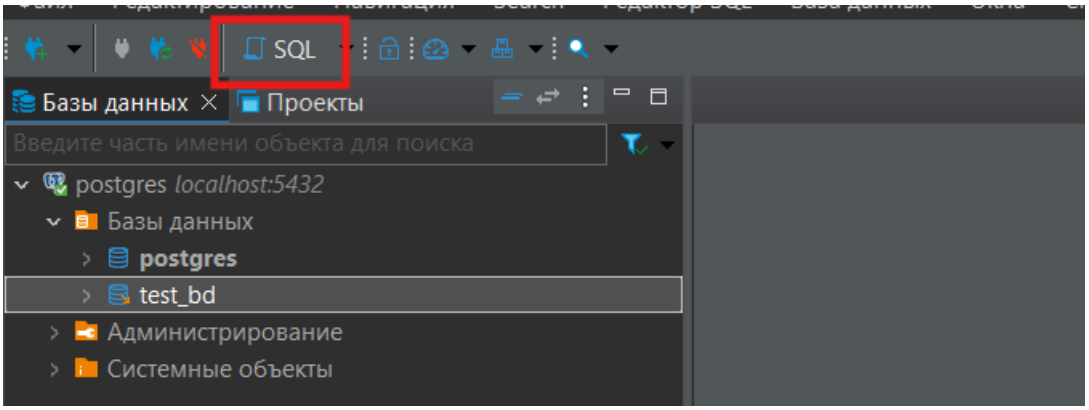


Создаём БД.



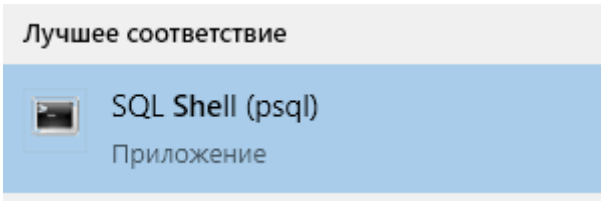


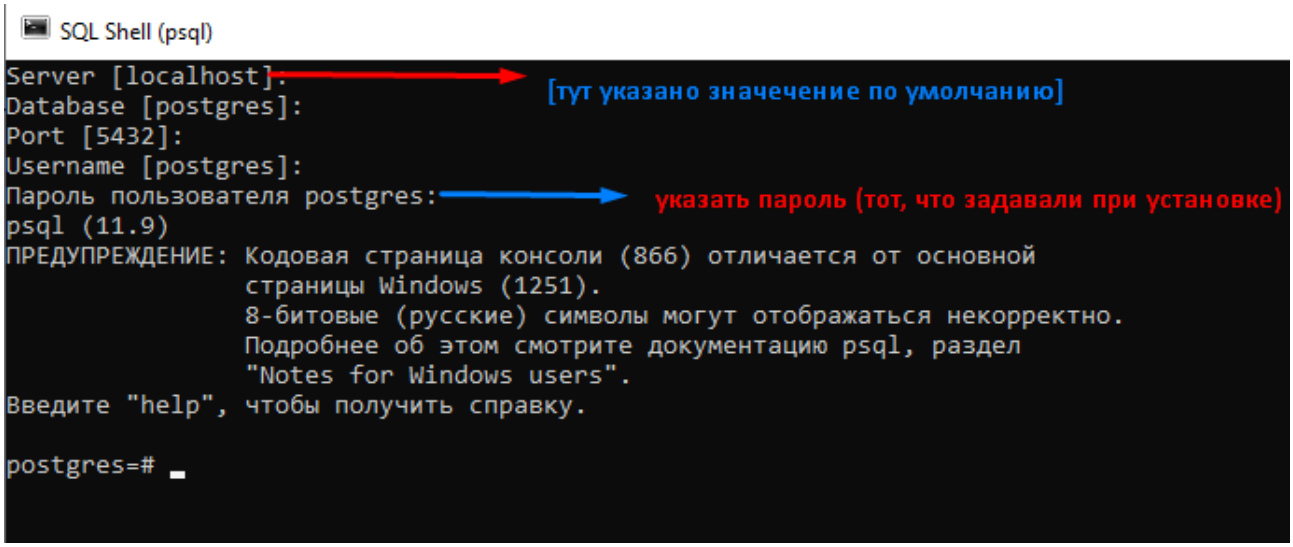
Открываем редактор SQL.



1.3 Создание БД в psql

Вместе с сервером PostgreSQL был установлен и консольный клиент. Нужно его запустить.





Создадим базу данных.

```
postgres=# create database fitness_db2;
CREATE DATABASE
postgres=#
```

Подключаемся к БД.

```
postgres=# \c fitness_db2
Вы подключены к базе данных "fitness_db2" как пользователь "postgres".
fitness_db2=#
```

Кодировка, установленная по умолчанию в консоли, не поддерживает отображение кириллицы. Чтобы исправить это, необходимо ввести команду \! chcp 1251

```
postgres=# \! chcp 1251
```

1.4 Язык SQL

SQL - простой язык программирования, который имеет немного команд и которой может научиться любой желающий. Расшифровывается как Structured Query Language - язык структурированных запросов, который был разработан для работы с БД, а именно, чтобы получать /добавлять /изменять данные, иметь возможность обрабатывать большие массивы информации и быстро получать структурированную и сгруппированную информацию. Есть много вариантов языка SQL, но у них всех основные команды почти одинаковы.

Язык SQL разделяется на 4 группы, описание которых с примерами команд указано в таблице ниже.

Группа	Краткое описание	Примеры команд
DDL	Группа операторов определения данных. Другими словами, с помощью операторов, входящих в эту группы, мы определяем структуру базы данных и работаем с объектами этой базы, т.е. создаем, изменяем и удаляем их.	CREATE ALTER DROP Эти операторы мы используем в ЛР3 и ЛР4.
DML	Группа операторов для манипуляции данными. С помощью этих операторов мы можем добавлять, изменять, удалять и выгружать данные из базы, т.е. манипулировать ими.	SELECT INSERT UPDATE DELETE Эти операторы мы используем в ЛР3, ЛР4, ЛР5
DCL	Группа операторов определения доступа к данным. Иными словами, это операторы для управления разрешениями, с помощью них мы	GRANT REVOKE DENY Эти операторы не будут использоваться в рамках курса Управление данными

	можем разрешать или запрещать выполнение определенных операций над объектами базы данных.	
TCL	Группа операторов для управления транзакциями. Транзакция – это команда или блок команд (инструкций), которые успешно завершаются как единое целое, при этом в базе данных все внесенные изменения фиксируются на постоянной основе или отменяются, т.е. все изменения, внесенные любой командой, входящей в транзакцию, будут отменены.	BEGIN/COMMIT/ROLLBACK/SAVE TRANSACTION Эти операторы не будут использоваться в рамках курса Управление данными

2 СОЗДАНИЕ ТАБЛИЦ

Таблицы создаются с помощью CREATE-запросов. А если быть точнее - через CREATE TABLE.

Ссылка на документацию по CREATE TABLE:
https://postgrespro.ru/docs/postgresql/15/sql-createtable

Нужно начать создавать таблицы. Создать таблицы можно будет двумя способами.

Способ 1:

1. Сначала создать независимые таблицы. То есть те, в которых нет внешних ключей.
2. Создать таблицы, в которых есть внешние ключи для связи с уже созданными таблицами.
3. Создать оставшиеся таблицы.

Способ 2:

1. Создать все таблицы, но не указывать, какие столбцы являются внешними ключами.
2. Указать внешние ключи.

На схеме примера (тема «спортзал») независимые таблицы для удобства выделены голубым цветом.

Перед тем, как создавать таблицы, нужно немного забежать вперёд и рассказать про ограничения и операции ON DELETE и ON UPDATE в частности, чему посвящён следующий пункт.

2.1 Ограничения

Немного документации и примеров:
https://postgrespro.ru/docs/postgresql/15/ddl-constraints

Список ограничений представлен ниже.

Ограничение	Описание
PRIMARY KEY	Первичный ключ

FOREIGN KEY	Внешний ключ
NOT NULL	Обязательность заполнения столбца
UNIQUE	В столбце должны быть только уникальные значения
CHECK	Логическое условие (обычно простое)
DEFAULT	В столбце задано значение по умолчанию
Ограничение значений	Ограничение кол-во вводимых символов, численное ограничение Пример: varchar(20), numeric(10,2)
Ссылочные действия	ON DELETE/ON UPDATE - операции
Идентификатор	IDENTITY

Далее будет представлена информация про ссылочные ограничения (ON DELETE/ON UPDATE).

Ограничение	Описание
CASCADE	Каскадная операция (например, каскадное удаление или обновление) ON DELETE CASCADE - при удалении записи в родительской таблице удаляются связанные с ней записи в дочерней таблице ON UPDATE CASCADE - при обновлении ID записи в родительской таблице обновляется значение внешнего ключа в зависимой таблице
NO ACTION	ON DELETE/ON UPDATE NO ACTION - при удалении записи в родительской таблице или обновлении ID этой записи в дочерней таблице ничего не

	делать. Данное ограничение установлено по умолчанию.
SET NULL	ON DELETE SET NULL - при удалении записи в родительской таблице значение внешнего ключа связанной ранее с ней записи в дочерней таблице становится равным NULL. Работает только для необязательных для заполнения внешних ключей.
SET DEFAULT	ON DELETE SET NULL - при удалении записи в родительской таблице значение внешнего ключа связанной ранее с ней записи в дочерней таблице сбрасывается до значения по умолчанию. Работает только для столбцов, являющихся внешними ключами, для которых установлено ограничение DEFAULT.

Как вы знаете, таблицы связываются друг с другом по паре значений «первичный ключ» - «внешний ключ». То есть таблица с внешним ключом будет зависимой.

Могут возникнуть ситуации, когда в родительской таблице меняется значение первичного ключа (что бывает крайне редко) у какой-то записи, поэтому это значение нужно обновить во всех зависимых таблицах.

Могут возникнуть ситуации, когда в родительской таблице удаляется запись, поэтому нужно будет удалить все связанные с этой записью строки в зависимой таблице.

В СУБД PostgreSQL реализована возможность добавления ON DELETE и ON UPDATE - операций к внешним ключам.

Операции ON DELETE позволяют указывать, ЧТО делать с записью в ЗАВИСИМОЙ таблице при удалении связанной с ней записи в родительской таблице.

Операции ON UPDATE позволяют указывать, ЧТО делать с записью в ЗАВИСИМОЙ таблице при изменении значения первичного ключа в родительской таблице.

Список таких операций:

CASCADE: автоматическое удаление или изменение строки в зависимой таблице при удалении или изменении связанной строки в родительской таблице. Эта операция называется «каскадное удаление».

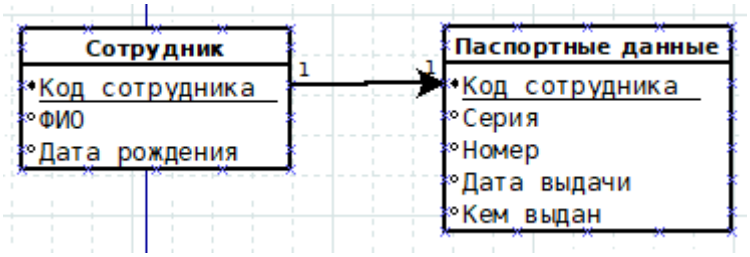
RESTRICT: запрет действий со строкой в зависимой таблице при удалении или изменении связанной строки в родительской таблице.

NO ACTION: действие по умолчанию, предотвращает какие-либо действия в зависимой таблице при удалении или изменении связанных строк в родительской таблице.

SET NULL: при удалении связанной строки из родительской таблицы устанавливает для столбца внешнего ключа значение NULL. В таком случае столбец, являющийся внешним ключом, не должен быть обязательным для заполнения.

SET DEFAULT: при удалении связанной строки из родительской таблицы устанавливает для столбца внешнего ключа значение по умолчанию, которое задается с помощью атрибута DEFAULT. Если для столбца не задано значение по умолчанию, то в качестве него применяется значение NULL. Соответственно, работает для тех столбцов, у которых есть DEFAULT-ограничение.

Например, есть такие таблицы.



В ней есть какие-то записи о сотрудниках и об их паспортных данных. Допустим, появилась необходимость удалить полностью информацию о сотруднике. Раз мы это делаем, то нет необходимости хранить его паспортные данные.

Если мы сразу начнём удалять данные из таблицы «Сотрудник», то столкнёмся с ошибкой. Эта ошибка будет говорить о том, что нельзя удалить запись, потому что есть

зависимые от неё записи в других таблицах. Для начала нужно будет удалить все зависимые записи, а только потом удалять родительскую.

Однако, благодаря операции ON DELETE можно указать, что делать при удалении и облегчить сам процесс удаления. В данном случае можно сделать так, чтобы при удалении записи из таблицы «Сотрудник» автоматически удалялась связанная запись в таблице «Паспортные данные».

Делается это так:

```
CREATE TABLE employees(
employee_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
fullname VARCHAR(100) NOT NULL,
birthdate DATE NOT NULL);

CREATE TABLE passport(
employee_id INT PRIMARY KEY, Кстати, тут не указываем генерацию значений, т.к. в таблице
p_series CHAR(5) NOT NULL,   первичный ключ одновременно является внешним из-за связи
p_number CHAR(6) NOT NULL,   между таблицами "один-к-одному"
p_issuedate DATE NOT NULL,
p_department VARCHAR(100) NOT NULL,
FOREIGN KEY (employee_id) REFERENCES employees(employee_id) ON DELETE CASCADE);
```

Можно заметить, что к внешнему ключу во втором запросе добавляется ON DELETE CASCADE.

Именно это позволит при удалении записи из таблицы «Сотрудник» удалить связанную с ней запись в таблице «Паспортные данные».

Допустим, столбец «employee_id» в таблице «Сотрудники» не генерируется автоматически. И табельный номер по каким-то причинам может быть изменён. В таком случае при изменении табельного номера нужно его поменять во всех связанных таблицах. И запросы будут выглядеть так:

```
CREATE TABLE employees(
employee_id INT PRIMARY KEY,
fullname VARCHAR(100) NOT NULL,
birthdate DATE NOT NULL);

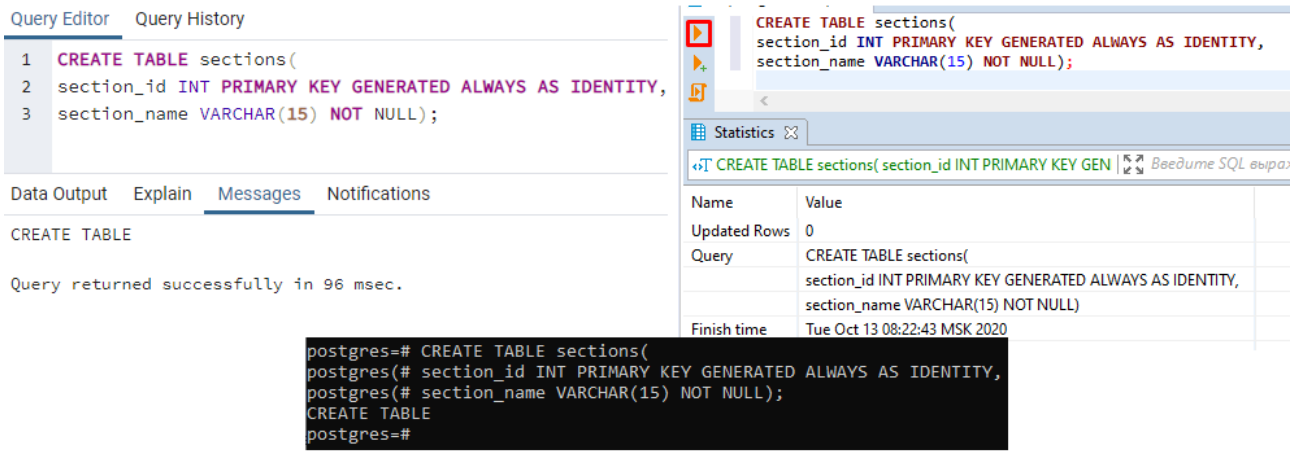
CREATE TABLE passport(
employee_id INT PRIMARY KEY,
p_series CHAR(5) NOT NULL,
p_number CHAR(6) NOT NULL,
p_issuedate DATE NOT NULL,
p_department VARCHAR(100) NOT NULL,
FOREIGN KEY (employee_id) REFERENCES employees(employee_id) ON UPDATE CASCADE);
```

Нужно отметить, что ON UPDATE CASCADE («каскадное обновление») применяется крайне редко, потому значения первичных ключей обычно не меняются. Да и вообще не так часто возникают ситуации, когда первичный ключ - это не генерируемое автоматически значение.

2.2 Создание таблицы «Секция»



Таблица	Столбец	Обязательность	Тип данных	Ограничения	Описание
Секции	Код секции	Да	INT с IDENTITY	Первичный ключ. Автоматическое увеличение на 1.	Идентификатор секции
	Название секции	Да	VARCHAR(15)	-	Имя секции



Разберём скрипт.

```
CREATE TABLE sections(
section_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
section_name VARCHAR(15) NOT NULL);
```

CREATE TABLE - команда для создания таблицы.
sections - имя таблицы.
() - границы структуры таблицы.
section_id, section_name - имена столбцов.
INT, VARCHAR(15) - типы данных столбцов.
PRIMARY KEY - указывает, что этот столбец является первичным ключом.
GENERATED ALWAYS AS IDENTITY - указывает, что этот столбец будет являться «идентификатором» - значения этого столбца будут браться из последовательности, и по умолчанию каждое новое значение будет больше предыдущего на 1.
NOT NULL - указывает, что столбец является обязательным заполнения.

Первичный ключ можно задать иначе.

```
CREATE TABLE sections(
section_id INT GENERATED ALWAYS AS IDENTITY,
section_name VARCHAR(15) NOT NULL,
PRIMARY KEY(section_id));
```

здесь первичный ключ задаётся после определения всех столбцов

Или так.

```
CREATE TABLE sections(
section_id INT GENERATED ALWAYS AS IDENTITY,
section_name VARCHAR(15) NOT NULL);

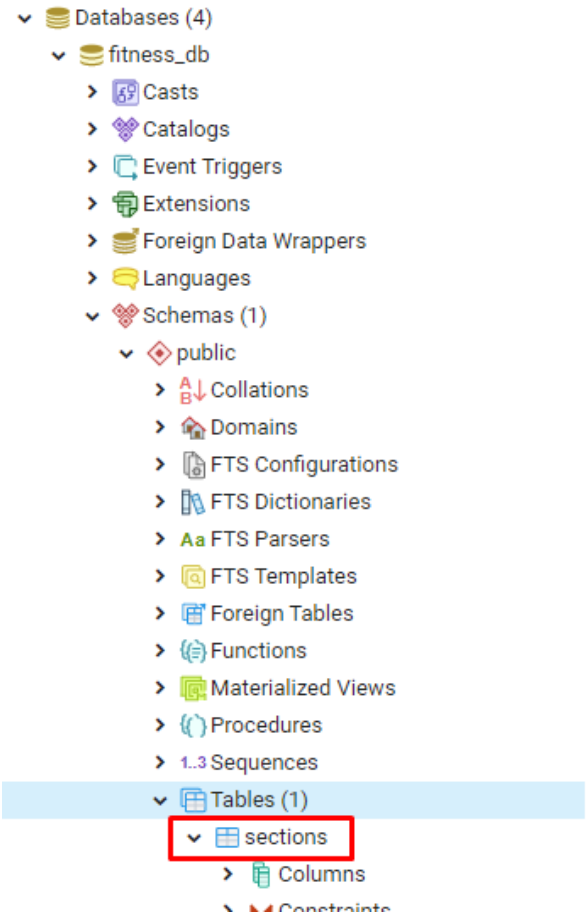
ALTER TABLE sections ADD PRIMARY KEY(section_id);
```

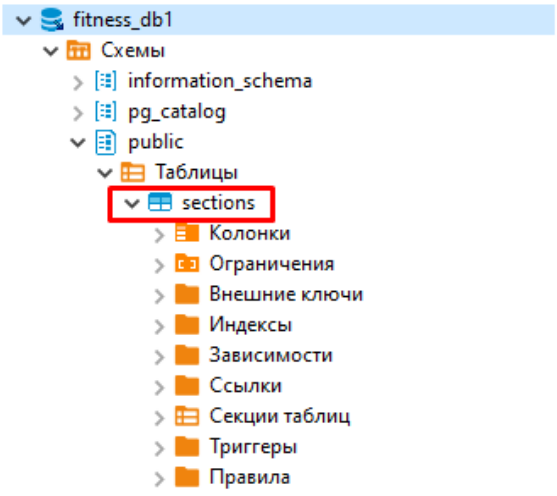
В этом случае сначала создаётся таблица без ключей. А ключ добавляется в другом запросе.

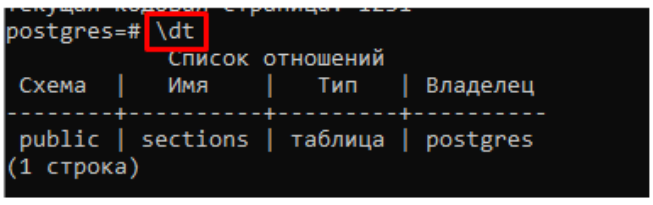
Разберём второй скрипт.

ALTER TABLE - команда изменения структуры таблицы.
sections - имя изменяемой таблицы.
ADD PRIMARY KEY - после ADD указывается, что добавляется к таблице. В данном случае добавляется первичный ключ.
(section_id) - указывает, что за столбец устанавливается в качестве первичного ключа.

Просмотреть список таблиц можно так:







Так как процесс создания большинства таблиц похож друг на друга, то будет рассмотрено только несколько случаев, чтобы разобрать синтаксические особенности.

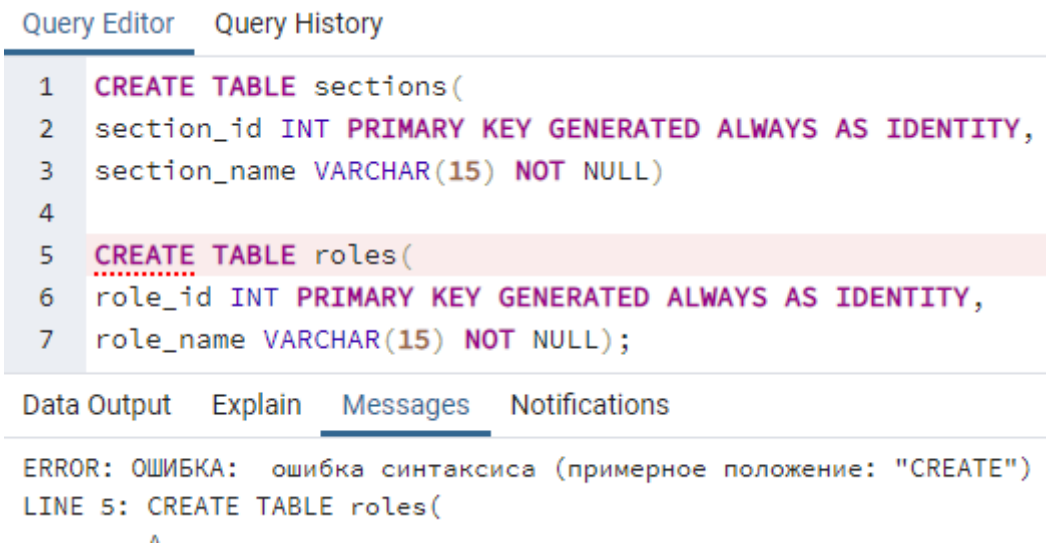
Написание скриптов

Вне зависимости от редактора скрипты нужно разделять символом «;» («точка с запятой»).

```
CREATE TABLE sections(
section_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
section_name VARCHAR(15) NOT NULL);

CREATE TABLE roles(
role_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
role_name VARCHAR(15) NOT NULL);
```

Если убрать точку с запятой после первого запроса, то будет следующая ошибка



[Про pgAdmin:]

Скрипты в одном файле выполняются последовательно друг за другом. Если выщестоящий скрипт не выполнился, то все остальные скрипты не запустятся.

Что касается скриптов создания таблиц, то если одна таблица уже была создана, то повторное выполнение скрипта создания таблицы приведёт к ошибке с текстом о том, что таблица уже существует.

Query Editor Query History

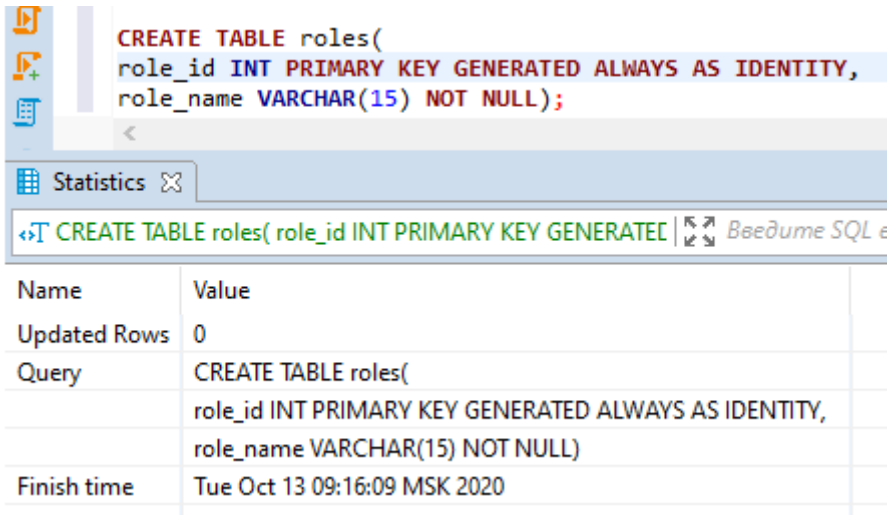
```
1  CREATE TABLE sections(
2  section_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
3  section_name VARCHAR(15) NOT NULL);
4
5  CREATE TABLE roles(
6  role_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
7  role_name VARCHAR(15) NOT NULL);
```

Data Output Explain Messages Notifications

```
ERROR: ОШИБКА:  отношение "sections" уже существует
```

Если необходимо выполнить какой-то отдельный скрипт, то его нужно выделить.

[Про dbeaver:]
В dbeaver’е, как и во многих других SQL-клиентах можно не выделять запрос, а просто перейти на одну из его строк.



2.3 Создание таблицы «Типы товаров»

Создадим таблицу «Типы товаров».

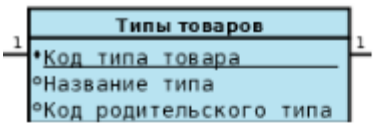


Таблица	Столбец	Обязательность	Тип данных	Ограничения	Описание
Типы товаров	Код типа товара	NOT NULL	INT с IDENTITY	Первичный ключ. Автоматическое увеличение на 1.	Идентификатор типа товара
	Название типа	NOT NULL	VARCHAR(30)	-	Имя типа товара
	Код родительского типа	NULL	INT	-	Идентификатор родителя типа товара

```
CREATE TABLE product_types(
type_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
type_name VARCHAR(30) NOT NULL,
parent_type_id INT);
```

Столбец необязательный, поэтому NOT NULL писать не надо

2.4 Создание таблицы «Инвентарь»

Создадим таблицу «Инвентарь». В этой таблице есть один внешний ключ «Код типа инвентаря». Он ссылается на уже созданную в предыдущем пункте таблицу «Типы товаров».

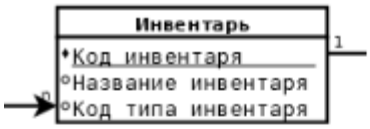


Таблица	Столбец	Обязательность	Тип данных	Ограничения	Описание
Инвентарь	Код инвентаря	Да	INT с IDENTITY	Первичный ключ. Автоматическое увеличение на 1.	Идентификатор инвентаря
	Название инвентаря	Да	VARCHAR(30)	-	Имя инвентаря
	Код типа инвентаря	Да	INT	Внешний ключ для связи с таблицей “Типы товаров”.	Идентификатор типа инвентаря. Ссылка на таблицу “Типы товаров”.

```
CREATE TABLE inventory(
inventory_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
inventory_name VARCHAR(30) NOT NULL,
product_type_id INT NOT NULL,
FOREIGN KEY (product_type_id) REFERENCES product_types(type_id));
```

Разберём последнюю строчку:

FOREIGN KEY - указание на внешний ключ.

(product_type_id) - в скобках указывается столбец таблицы, который будет являться внешним ключом.

REFERENCES - оператор добавления ссылки.

product_types(type_id) - таблица, на которую создаётся ссылка(столбец, на который создаётся ссылка).

Добавить внешний ключ можно иным способом.

```
ALTER TABLE inventory
    ADD CONSTRAINT product_type_id_fk
        FOREIGN KEY (product_type_id) REFERENCES product_types(type_id);
```

ADD CONSTRAINT - добавление ограничения.

product_type_id_fk - имя ограничения.

FOREIGN KEY - указание на внешний ключ.

(product_type_id) - в скобках указывается столбец таблицы, который будет являться внешним ключом.

REFERENCES - оператор добавления ссылки.

product_types(type_id) - таблица, на которую создаётся ссылка(столбец, на который создаётся ссылка).

2.5 Создание таблицы «Состав секций»

Создадим таблицу «Состав секций». В ней есть два внешних ключа.

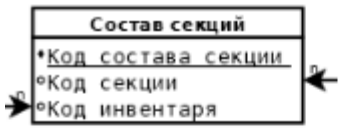


Таблица	Столбец	Обязательность	Тип данных	Ограничения	Описание
Состав секций	Код состава секции	Да	INT с IDENTITY	Первичный ключ. Автоматическое увеличение на 1.	Идентификатор состава секции
	Код секции	Да	INT	Внешний ключ для связи с таблицей “Секции”.	Идентификатор секции. Ссылка на таблицу “Секции”.
	Код инвентаря	Да	INT	Внешний ключ для связи с таблицей “Инвентаря”.	Идентификатор инвентаря. Ссылка на таблицу “Инвентарь”.

```
CREATE TABLE section_composition(
composition_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
section_id INT NOT NULL,
inventory_id INT NOT NULL,
FOREIGN KEY (section_id) REFERENCES sections(section_id),
FOREIGN KEY (inventory_id) REFERENCES inventory(inventory_id));
```

2.6 Создание таблицы «Услуга»

Создадим таблицу «Услуга».

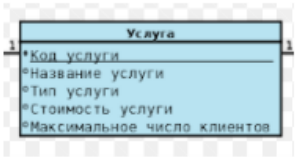


Таблица	Столбец	Обязательность	Тип данных	Ограничения	Описание
Услуга	Код услуги	Да	INT с IDENTITY	Первичный ключ. Автоматическое увеличение на 1.	Идентификатор услуги
	Название услуги	Да	VARCHAR(30)	-	Имя услуги
	Тип услуги	Да	VARCHAR(15)	Возможные значения: “персональная”, “групповая”	Тип услуги.
	Стоимость услуги	Нет	NUMERIC(10, 2)	Значение >0. Если не заполнено, то услуга бесплатная.	Стоимость услуги в рублях.
	Максимальное число клиентов	Нет	INT	Значение >0. Если не заполнено, то это не групповое занятие.	Возможное число клиентов.

```
CREATE TABLE service(
service_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
service_name VARCHAR(30) NOT NULL,
service_type VARCHAR(15) NOT NULL CHECK(service_type in ('персональная','групповая')),
service_cost NUMERIC(10,2) CHECK(service_cost > 0),
max_clients_number INT CHECK(max_clients_number > 0));
```

В данном запросе задаются три ограничения - проверки (CHECK).

Такие ограничения нужно задавать, когда значение в столбце должно соответствовать какому-то логическому выражению.

Ограничение CHECK (service_type in (‘персональная’, ‘групповая’) говорит о том, что в столбце service_type (тип услуги) может быть только одно из двух значений.

Ограничение CHECK (service_cost > 0) говорит о том, что стоимость услуги не может быть отрицательной или нулевой.

Подробности про данный тип ограничений можно узнать здесь:

https://postgrespro.ru/docs/postgresql/15/ddl-constraints

Там же можно найти больше примеров использования.

2.7 Создание таблицы «Сотрудники»

Создадим таблицу «Сотрудники».

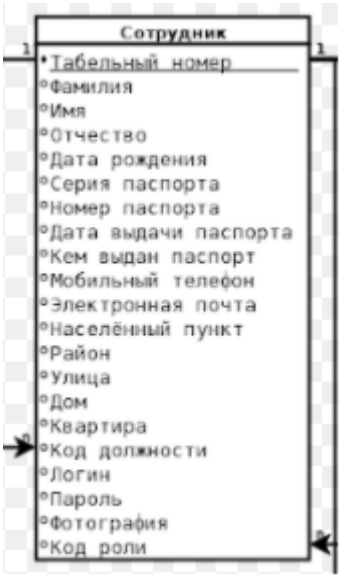


Таблица	Столбец	Обязательность	Тип данных	Ограничения	Описание
Сотрудник	Табельный номер	Да	INT с IDENTITY	Первичный ключ. Автоматическое увеличение на 1.	Идентификатор сотрудника
	Фамилия	Да	VARCHAR(30)	-	Фамилия сотрудника
	Имя	Да	VARCHAR(30)	-	Имя сотрудника
	Отчество	Нет	VARCHAR(30)	-	Отчество сотрудника
	Дата рождения	Да	DATE	Формат: DD.MM.YYYY	Дата рождения сотрудника
	Серия	Да	CHAR(5)	Формат:	Серия

	паспорта			СС СС	паспорта сотрудника
	Номер паспорта	Да	CHAR(6)	Формат: НННННН	Номер паспорта сотрудника
	Дата выдачи паспорта	Да	DATE	Формат: DD.MM.YYYY	Дата выдачи паспорта сотрудника
	Кем выдан паспорт	Да	VARCHAR(100)	-	Подразделение, выдавшее паспорт сотруднику
	Мобильный телефон	Да	CHAR(11)	Формат: 79991112233 Без других символов.	Контактный мобильный телефон сотрудника.
	Электронная почта	Да	VARCHAR(255)	Формат: название@домен.домен	Электронная почта.
	Населённый пункт	Да	VARCHAR(50)	-	Населённый пункт (город, село и т.п.) адреса фактического проживания сотрудника
	Район	Нет	VARCHAR(50)	-	Район (не внутри населённого пункта)
	Улица	Да	VARCHAR(50)	-	Улица
	Дом	Да	VARCHAR(4)	-	Дом (в т.ч. литер)
	Квартира	Нет	VARCHAR(5)	-	№ квартиры или офиса
	Код должности	Да	INT	Внешний ключ для связи с таблицей «Должности»	Идентификатор должности сотрудника. Ссылка на запись в таблице «Должности».

	Логин	Да	VARCHAR(255)	Строка в нижнем регистре, состоящая из символов латиницы и цифр.	Логин сотрудника для авторизации в приложении
	Пароль	Да	VARCHAR(255)	Строка, состоящая из символов латиницы и цифр.	Пароль учётной записи сотрудника для авторизации в приложении
	Фотография	Да	VARCHAR(255)	-	Ссылка на фотографию сотрудника, размещённую на сервере
	Код роли	Да	INT	Внешний ключ для связи с таблицей «Роли»	Идентификатор роли. Ссылка на запись в таблице «Роли». Возможно назначить только роли, отличные от «клиент».

```sql
CREATE TABLE employee(
employee_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
surname VARCHAR(30) NOT NULL,
firstname VARCHAR(30) NOT NULL,
middlename VARCHAR(30),
birthdate DATE NOT NULL,
passport_series CHAR(5) NOT NULL,
passport_number CHAR(6) NOT NULL,
passport_issuedate DATE NOT NULL,
passport_issuedepartment VARCHAR(100) NOT NULL,
phone CHAR(11) NOT NULL,
email VARCHAR(255) NOT NULL,
locality VARCHAR(50) NOT NULL,
district VARCHAR(50) NOT NULL,
street VARCHAR(50) NOT NULL,
house VARCHAR(4) NOT NULL,
room VARCHAR(5),
position_id INT NOT NULL,
login VARCHAR(255) NOT NULL,
pass VARCHAR(255) NOT NULL,
photo VARCHAR(255) NOT NULL,
role_id INT NOT NULL,
FOREIGN KEY (position_id) REFERENCES positions(position_id),
FOREIGN KEY (role_id) REFERENCES roles(role_id));
```

В этом запросе в принципе нет ничего особенного.

Формат: 79991112233 Без других символов.
Формат: название@домен.домен

Формат: DD.MM.YYYY	Д р с
Формат: CC CC	С п с
Формат: HHHHHH	Н п с

Указанные выше ограничения формата не реализовываются на уровне базы данных. Они будут реализованы в будущем в приложении, которое будет работать с базой данных. Однако, важно понимать, как хранить данные ещё на этапе создания БД.

2.8 Создание таблицы «Товар»

Таблица	Столбец	Обязательность		Ограничения	Описание
Товар	Код товара	Да	INT с IDENTITY	Первичный ключ. Автоматическое увеличение на 1.	Идентификатор товара
	Наименование товара	Да	VARCHAR(50)	-	Имя товара
	Код типа товара	Да	INT	Внешний ключ для связи с таблицей “Типы товаров”.	Идентификатор кода типа товара. Ссылка на таблицу “Типы товаров”.
	Стоимость закупки	Да	NUMERIC(10, 2)	Значение >0.	Стоимость закупки товара у поставщика в рублях.
	Стоимость продажи	Да	NUMERIC(10, 2)	Значение >0. Значение должно быть больше стоимости закупки.	Стоимость продажи товара клиентам.
	Количество	Да	INT	Значение >=0.	Количество товара на складе.
	Фотография	Нет	INT	-	Фото товара.

```
CREATE TABLE product(
product_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
product_name VARCHAR(50) NOT NULL,
product_type_id INT NOT NULL,
cost_in NUMERIC(10,2) NOT NULL CHECK(cost_in > 0),
cost_out NUMERIC(10,2) NOT NULL CHECK(cost_out > cost_in),
product_count INT NOT NULL CHECK(product_count > 0),
product_photo INT);
```

Здесь есть другой способ использования ограничения CHECK: CHECK(cost_out > cost_in). В этом случае мы указываем, что значение в столбце «Стоимость продажи» должно быть больше значения столбца «Стоимость закупки».

2.9 Создание таблицы «Заказы»

Таблица	Столбец	Обязательность	Тип данных	Ограничения	Описание
Заказы	Код заказа	Да	INT с IDENTITY	Первичный ключ. Автоматическое увеличение на 1.	Идентификатор заказа
	Код клиента	Да	INT	Внешний ключ для связи с таблицей “Клиент”	Идентификатор клиента. Ссылка на таблицу “Клиент”.
	Табельный номер	Да	INT	Внешний ключ для связи с таблицей “Сотрудник”.	Идентификатор сотрудника, проводящего продажу. Ссылка на таблицу “Сотрудник”.
	ДатаВремя продажи	Да	TIMESTAMP	Формат: DD.MM.YYYY HH:MM:SS	Дата и время осуществления транзакции
	Сумма заказа	Да	NUMERIC(10, 2)	Значение >= 0. Значение по умолчанию = 0.	Сумма всех товаров в заказе + сумма доставки.

```
CREATE TABLE orders(
order_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
client_id INT NOT NULL,
employee_id INT NOT NULL,
order_datetime TIMESTAMP NOT NULL,
order_sum NUMERIC(10,2) NOT NULL CHECK(order_sum >=0) DEFAULT 0,
FOREIGN KEY (client_id) REFERENCES clients(client_id),
FOREIGN KEY (employee_id) REFERENCES employee(employee_id));
```

Здесь применяется ограничение «значение по умолчанию»: DEFAULT 0.

3 ИЗМЕНЕНИЕ ТАБЛИЦ

Изменение таблиц производится с помощью запросов ALTER TABLE.

Ссылка на документацию по ALTER TABLE: https://postgrespro.ru/docs/postgresql/15/sql-altertable

3.1 Переименование таблицы

Переименуем таблицу «Товар».
Сейчас она называется «product». Сделаем «products».

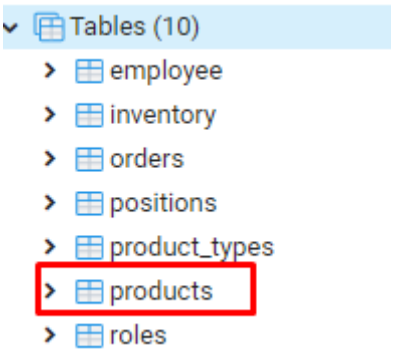
```
ALTER TABLE product RENAME TO products;
            старое имя          новое имя
```

ata Output | Explain | Messages | Notifications

.TER TABLE

product - старое имя таблицы;
RENAME TO - указание на изменение имени;
products - новое имя таблицы.

Посмотреть изменения можно в списке таблиц.



3.2 Добавление столбцов

Будем работать с таблицей «Секции».

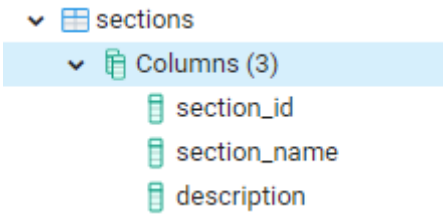
```
CREATE TABLE sections(
section_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
section_name VARCHAR(15) NOT NULL);
```

Добавим в неё тестовый столбец.

```
ALTER TABLE sections ADD COLUMN description varchar(100);
```

sections - имя изменяемой таблицы;
ADD COLUMN - операция по добавлению столбца;
description - имя добавляемого столбца;
varchar(100) - тип данных добавляемого столбца.

Результат:



Если у вас в таблице уже были какие-то данные, а вы добавляете столбцы, то если они требуют заполнения сразу, то обязательно сразу же это и надо сделать.

Новые столбцы могут быть связаны с каким-то новым функционалом и будут заполняться по мере необходимости при работе пользователей с приложением.

Можно сразу же добавляемый столбец сделать первичным ключом. Добавлять ограничения и т.д. Пример будет показан на тестовой таблице.

```
CREATE TABLE test_table1(
test_int_column INT,
test_text_column VARCHAR(30));
```

```
ALTER TABLE test_table1 ADD COLUMN test_id_pk INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY;
```

Данный запрос добавляет в таблицу первичный ключ с типом INT и автоматической генерацией значений.

3.3 Удаление столбцов

По своей логике удаление столбцов - более сложная операция нежели добавление.

На удаление столбцов влияет:

- Наличие или отсутствие связи с другими таблицами (т.е. является ли столбец внешним ключом);
- Наличие данных в столбце.

Ниже показан пример удаления добавленного в пункте 3.2 столбца.

```
ALTER TABLE sections DROP COLUMN description;
```

sections - имя изменяемой таблицы;
DROP COLUMN - команда удаления столбца;
description - имя удаляемого столбца.

Аналогичным образом удаляются столбцы с данными и столбцы, являющиеся внешними ключами.

Однако, в этом случае нужно предварительно ответить на следующие вопросы: «следует ли мне сохранить данные столбца?», «следует ли вообще удалять этот столбец?», «не придётся ли затем этот столбец возвращать?», «не будет ли нарушена логика БД и работы приложения, если я удалю этот столбец?». Некоторые вопросы особенно актуальны, если вы удаляете внешний ключ.

3.4 Переименование столбцов

Переименуем столбец «description» в «descr».

```
ALTER TABLE sections RENAME COLUMN description TO descr;
```

RENAME COLUMN - команда переименования столбца;

description - старое имя столбца;
descr - новое имя столбца.

3.5 Изменение типов данных столбцов

Изменить тип данных у столбца - дело простое, если в нём нет данных. Если данные в столбце есть, то всё усложняется.

Пример изменения типа данных, когда столбец не заполнен ни в одной строке таблицы. Далее будут показаны лишь примеры. Не нужно задаваться вопросом «зачем мы меняем тип данных для столбца с описанием на численный?». Это просто пример.

```
select * from sections;
```

ata Output | Explain | Messages | Notifications

section_id [PK] integer	section_name character varying (15)	descr character varying (100)
1	Зелёная	[null]

3.5.1 Расширение и сужение столбцов

Для начала «расширим» столбец до 255 символов.

```
ALTER TABLE sections ALTER COLUMN description TYPE VARCHAR(255);
```

ALTER COLUMN - команда изменения столбца;
description - имя изменяемого столбца;
TYPE - указание на изменение типа данных;
VARCHAR(255) - новый тип.

Рассмотрим ситуацию, при которой в столбце есть данные, но нам необходимо, наоборот, сузить размер столбца.

Data Output | Explain | Messages | Notifications

	section_id [PK] integer	section_name character varying (15)	description character varying (255)
1	1	Зелёная	[null]
2	2	Красная	Какое-то описание

Попробуем сузить размер столбца до 10 символов. В этом случае нарвёмся на ошибку. Справедливо.

```
111  ALTER TABLE sections ALTER COLUMN description TYPE VARCHAR(10);
112
```

Data Output | Explain | Messages | Notifications

```
ERROR: ОШИБКА:  значение не умещается в тип character varying(10)
```

3.5.2 Изменение типов данных

Попробуем изменить тип данных. У столбца был тип данных varchar. Меняем на «родственный» тип данных char.

```
ALTER TABLE sections ALTER COLUMN description TYPE CHAR(30);
```

Изменим обратно.

```
115  ALTER TABLE sections ALTER COLUMN description TYPE VARCHAR(255);
116
```

Data Output | Explain | Messages | Notifications

```
ALTER TABLE
```

Заполним столбец следующими данными.

Data Output | Explain | Messages | Notifications

	section_id [PK] integer	section_name character varying (15)	description character varying (255)
1	1	Зелёная	1
2	2	Красная	2

В столбце «description» хранятся числа. Но в виде строк.

Попробуем изменить тип данных на INT. Ничего не предвещает беды.

```
120  ALTER TABLE sections ALTER COLUMN description TYPE INT;
121
```

Data Output | Explain | Messages | Notifications

```
ERROR: ОШИБКА:  столбец "description" нельзя автоматически привести к типу integer
HINT:  Возможно, необходимо указать "USING description::integer".
```

Для того, чтобы изменить тип данных, нужно воспользоваться специальными функциями для приведения типов.

```
ALTER TABLE sections ALTER COLUMN description TYPE INT USING (description::INT);
```

В этом случае используется предложение USING, внутри которого осуществляется преобразование типа столбца в INT.

:: - сокращённая запись функции CAST, осуществляющей преобразование типа.

```
USING (CAST(description as INT));
```

Однако, этот способ будет работать только при условии, что данные в столбцах соответствуют формату типа, в который осуществляется преобразование. Например, если в столбец записать нецелое значение и попытаться привести тип данных в INT, то мы столкнёмся с ошибкой.

Data Output | Explain | Messages | Notifications

```
ERROR: ОШИБКА:  неверное значение для целого числа: "2.3"
```

3.6 Работа с ограничениями

С помощью ALTER-запросов можно добавлять, изменять и задавать ограничения.

Работа будет вестись на тестовой таблице, которая хранит список тестов.

```
CREATE TABLE test2(
test_id INT,
test_name VARCHAR(30),
test_description VARCHAR(255) NOT NULL,
author_id INT NOT NULL,
raiting FLOAT NOT NULL);
```

```
CREATE TABLE author(
author_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
author_name VARCHAR(30) NOT NULL);
```

test_id - идентификатор теста.
test_name - имя теста.
test_description - описание теста.
author_id - идентификатор автора теста.
raiting - рейтинг теста.
author_name - имя автора теста.

3.6.1 Изменение обязательности

Столбец «test_name» сейчас является необязательным для заполнения. То есть можно не заполнять название теста, что является ошибкой проектирования БД. Исправим.

```
144
145  ALTER TABLE test2 ALTER COLUMN test_name SET NOT NULL;
146
```

Data Output | Explain | Messages | Notifications

```
ALTER TABLE
```

Исправили.

После операции SET указываем то, что столбец теперь является обязательным для заполнения.

А вот столбец с описанием зачем-то сделали обязательным, хотя описание может отсутствовать. Уберём обязательность с этого столбца.

```
145  ALTER TABLE test2 ALTER COLUMN test_description DROP NOT NULL;
146
```

И эту проблему мы решили. Команда DROP нужна для удаления ограничения.

3.6.2 Добавление первичного ключа

Если при создании таблицы по каким-то причинам не был указан первичный ключ, то его можно добавить альтернативным способом (не удалять же таблицу и создавать её вновь? Хотя тоже можно).

Укажем в качестве первичного ключа столбец «test_id».

```
ALTER TABLE test2 ADD PRIMARY KEY (test_id);
```

3.6.3 Добавление внешнего ключа

С помощью ALTER-запросов можно добавлять внешние ключи. Укажем в качестве внешнего ключа столбец «author_id» и свяжем его с таблицей авторов.

```
ALTER TABLE test2 ADD CONSTRAINT author_id_fk FOREIGN KEY (author_id) REFERENCES author(author_id);
```

ADD CONSTRAINT - добавление ограничения.

author_id_fk - имя добавляемого ограничения. Имя нужно, например, для более быстрого изменения или удаления ограничения. Оно также позволяет быстрее искать ошибки при работе с таблицами.

Всё остальное вы уже знаете :)

3.6.4 Работа с другими ограничениями

Например, для столбца «raiting» нужно задать следующие ограничения: значение по умолчанию должно быть равно 0; значение должно быть больше либо равно 0.

```
ALTER TABLE test2 ALTER COLUMN raiting SET DEFAULT 0;
```

Этот запрос добавляет значение по умолчанию.

```
161  ALTER TABLE test2 ADD CONSTRAINT raiting_check CHECK (raiting >= 0);
```

Data Output | Explain | Messages | Notifications

```
ALTER TABLE
```

Этот запрос добавляет ограничение с именем «raiting_check», которое устанавливает проверку значений в столбце.

Если нужно удалить ограничение на значение по умолчанию, то можно будет сделать это следующим образом:

```
ALTER TABLE test2 ALTER COLUMN raiting DROP DEFAULT;
```

Если нужно изменить ограничение CHECK (например, значение в столбце «raiting» должно быть строго больше 0), то нужно будет сначала удалить это ограничение, а затем добавить новое. Команда ALTER CONSTRAINT существует, но работает только для внешних ключей.

4 УДАЛЕНИЕ ТАБЛИЦ

Удаление таблиц осуществляется с помощью команды DROP TABLE.

Ссылка на документацию:
https://postgrespro.ru/docs/postgresql/15/sql-droptable

Синтаксис такой команды представлен ниже:

```
165  DROP TABLE author;
```

Порядок удаления таблиц:
1. Удаляем таблицы с внешними ключами, от которых не зависят никакие другие таблицы.
2. Удаляем таблицы без внешних ключей.

Если шаг 2 выполнить раньше, то будет такая ошибка:

```
165  DROP TABLE author;
166
```

Data Output | Explain | Messages | Notifications

```
ERROR: ОШИБКА:  удалить объект таблица author нельзя, так как от него зависят другие объекты
DETAIL:  ограничение author_id_fk в отношении таблица test2 зависит от объекта таблица author
HINT:  Для удаления зависимых объектов используйте DROP ... CASCADE.
```

```
165  DROP TABLE author;
166
```

Data Output | Explain | Messages | Notifications

```
DROP TABLE

Query returned successfully in 53 msec.
```

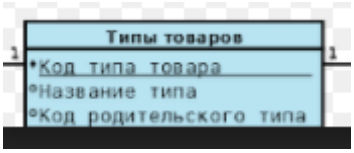
Можно, конечно, пойти и иным путём. И выполнить команду DROP TABLE имя_таблицы CASCADE. В этом случае вместе с указанной таблицей будут удалены зависимые от неё таблицы, если в свою очередь от этих таблиц ничего не зависит.

5 ЗАПОЛНЕНИЕ ТАБЛИЦ

Таблицы заполняются с помощью INSERT-запросов.
Ссылка на документацию:
https://postgrespro.ru/docs/postgresql/15/sql-insert

Порядок заполнения таблиц:
1. Заполняем независимые таблицы;
2. Заполняем остальные таблицы.

Для примера добавим несколько записей в таблицу «Типы товаров».




```
CREATE TABLE product_types(
type_id INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
type_name VARCHAR(30) NOT NULL,
parent_type_id INT);
```

В этой таблице значения первичного ключа генерируются автоматически.

Добавим следующие типы товаров:
1. Еда (родительская категория)
    1.1 Энергетический батончик
    1.2 Пищевая добавка
2. Тренажер

```
INSERT INTO product_types(type_name) values('Еда');
```

INSERT INTO - запрос на добавление новой строки.
product_types - имя таблицы, в которую добавляется строка.
(скобки после имени таблицы) - здесь перечисляем столбцы, которые нужно заполнять. Если значение первичного ключа генерируется автоматически или же мы не хотим заполнять необязательный столбец, то эти столбцы указывать в

списке не нужно. В данном случае первичный ключ «type_id» генерируется, поэтому его не указываем. Значение добавится автоматически. Т.к. мы добавляем категорию товара, у которой нет родителя, то заполнять последний столбец не нужно.

values(значения столбцов) - в скобках указываем то, что хотим добавить.

Посмотреть данные в таблицах можно с помощью следующего запроса. Процесс получения данных из таблиц называется выборкой.

```
3  SELECT * FROM product_types;
4
```

Data Output | Explain | Messages | Notifications

	type_id [PK] integer	type_name character varying (30)	parent_type_id integer
1	1	Еда	[null]

select - имя команды выборки (получения) данных из таблицы.

* - указывает, что будем получать информацию из всех столбцов.

FROM - указывает, откуда именно будут выбираться данные.

product_types - имя таблицы, из которой производится выборка.

Подробно с этой командой мы познакомимся в следующей лабораторной работе. Не сейчас.

Мы добавили родительскую категорию «Еда». Добавим 2 подкатегории: «Энергетический батончик» и «Добавка».

Так как это подкатегории, то мы должны указать, какая категория является родительской.

Категория «Еда» имеет ID = 1, поэтому именно это значение мы должны указать в столбце «parent_type_id» при добавлении подкатегорий «Энергетический батончик» и «Добавка». Вот так это будет выглядеть.

```
3  SELECT * FROM product_types;
4
5  insert into product_types(type_name) values('Еда');
6  INSERT INTO product_types(type_name, parent_type_id) VALUES('Энергетический батончик',1);
7  INSERT INTO product_types(type_name, parent_type_id) VALUES('Добавка',1);
8
9  truncate table product_types RESTART IDENTITY;
```

Data Output | Explain | Messages | Notifications

type_id [PK] integer	type_name character varying (30)	parent_type_id integer
1	Еда	[null]
2	Энергетический батончик	1
3	Добавка	1

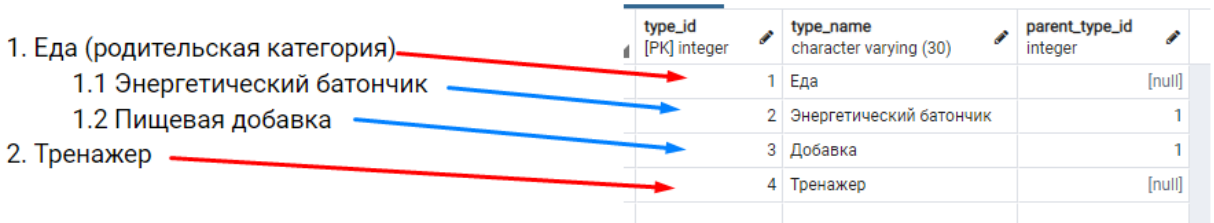
Добавим ещё одну категорию - «Тренажер».

```
3  SELECT * FROM product_types;
4
```

Data Output | Explain | Messages | Notifications

	type_id [PK] integer	type_name character varying (30)	parent_type_id integer
1	1	Еда	[null]
2	2	Энергетический батончик	1
3	3	Добавка	1
4	4	Тренажер	[null]

В итоге мы добавили немного иерархически связанных данных в таблицу.

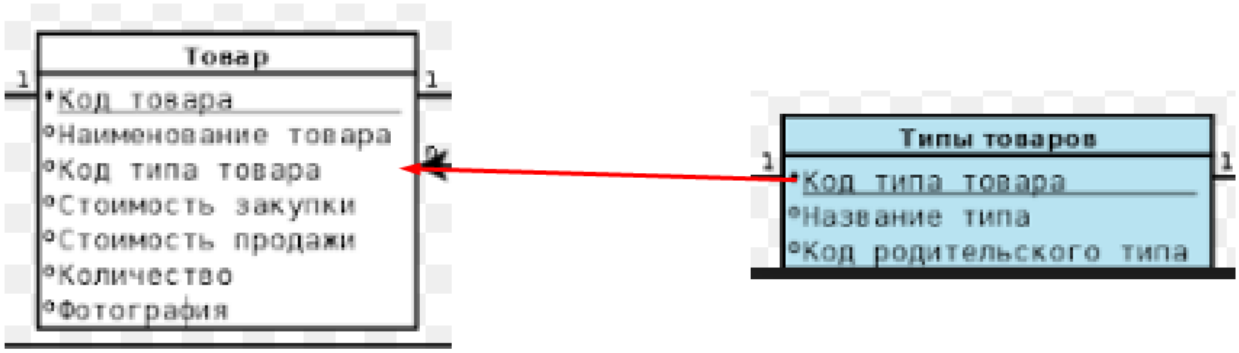


В каком порядке мы будем добавлять эти данные - неважно. Ещё абсолютно неважно, какое значение ID есть у той или иной записи.

Добавим несколько товаров, относящихся к разным категориям. Для удобства перед записью данных в таблицы

базы данных, мы должны понимать, какие данные мы будем записывать.

Если что, таблицы связаны таким образом.

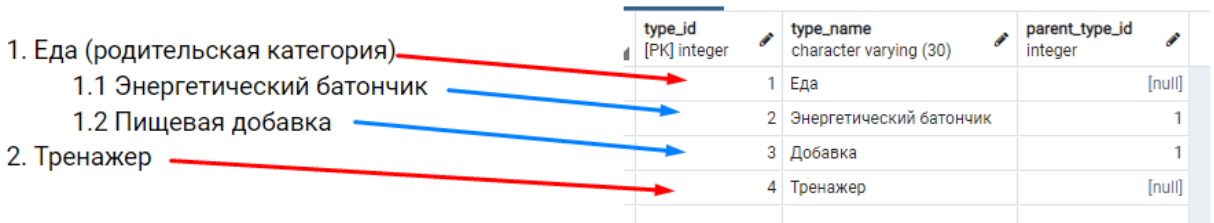


Добавим 2 батончика в таблицу с товарами.

Код товара	Имя товара	Стоимость закупки	Стоимость продажи	Количество	Фотография	Категория
1	Батончик глазированный "Bombbar" Ассорти, 30/40 г (30 порций)	1400	1600	5	-	Энергетический батончик
2	Батончик протеиновый "Bombbar", шоколад, фундук, 20 шт х 60 г (20 порций)	1700	1980	10	-	Энергетический батончик

Так как в таблице с товарами мы храним «Код типа товара», а не название категории, то мы должны названия заменить на соответствующие им коды товаров.

Если мы взглянем на это изображение...



...то увидим, что у категории товара с именем «Энергетический батончик» ID = 2.

В результате мы получим такие запросы:

```
11  select * from products;
12
13  INSERT INTO products(product_name, product_type_id, cost_in, cost_out, product_count)
14  VALUES('Батончик глазированный "Bombbar" Ассорти, 30/40 г (30 порций)', 2, 1400, 1600, 5);
15
16  INSERT INTO products(product_name, product_type_id, cost_in, cost_out, product_count)
17  VALUES('Батончик протеиновый "Bombbar", шоколад, фундук, 20 шт х 60 г (20 порций)', 2, 1700, 1980, 10);
18
```

Data Output | Explain | Messages | Notifications

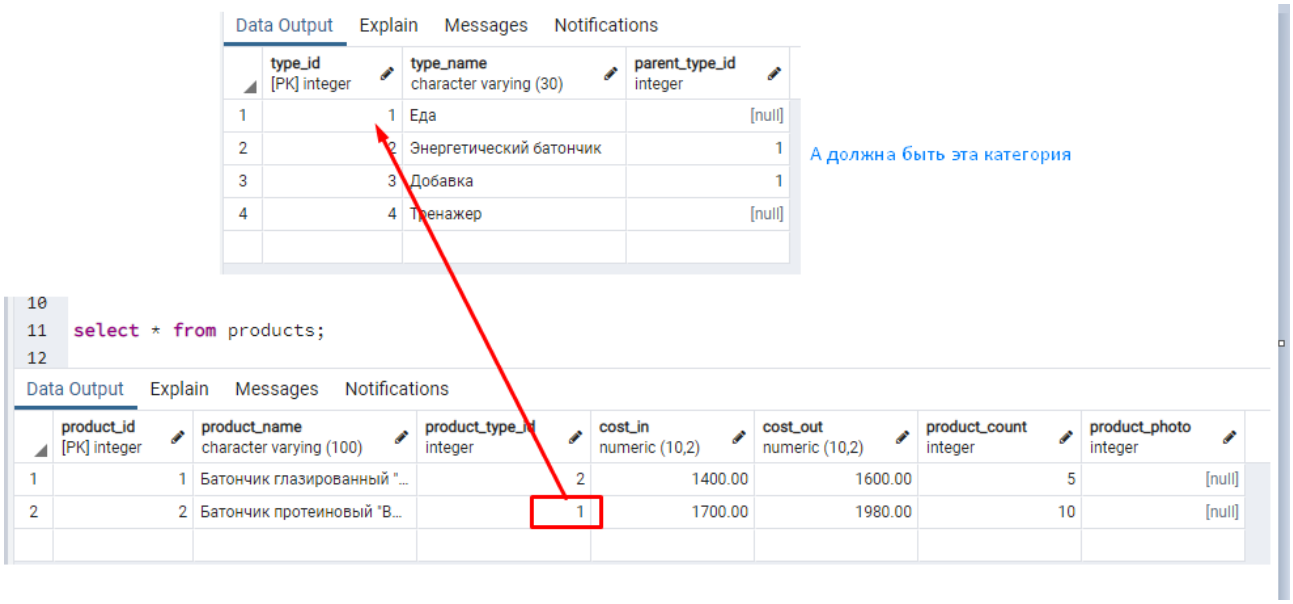
	product_id [PK] integer	product_name character varying (100)	product_type_id integer	cost_in numeric (10,2)	cost_out numeric (10,2)	product_count integer	product_photo integer
1	1	Батончик глазированный "...	2	1400.00	1600.00	5	[null]
2	2	Батончик протеиновый "B...	2	1700.00	1980.00	10	[null]

То, что мы указали «product_type_id» = 2, означает, что товары относятся к категории «Энергетический батончик».

6 ОБНОВЛЕНИЕ ДАННЫХ В ТАБЛИЦЕ

Одним из более популярных SQL-запросов является запрос, который меняет значение каких-либо данных в ячейках каких-либо таблиц.

Допустим, мы добавили товар в таблицу, но ошиблись с категорией товара.

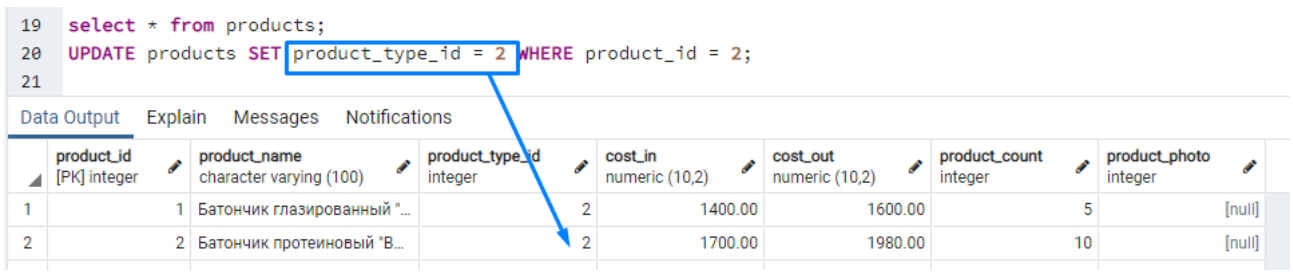


Мы должны это поправить! И не должны удалять запись и пересоздавать. А вдруг от этой записи зависят строки в других таблицах?

Изменение данных осуществляется с помощью UPDATE-запросов.

Ссылка на документацию:
https://postgrespro.ru/docs/postgresql/15/sql-update

Изменим категорию товара с «Еда» на «Энергетический батончик».



UPDATE - команда обновления.

products - таблица, в которой будет производиться изменение данных.

SET - операция, после которой мы указываем, что должно быть изменено.

WHERE - оператор, после которого мы указываем условия изменения данных.

product_id = 2 - условие изменение данных.

Этот запрос записывает в столбец «product_type_ud» значение 2 в строке с product_id = 2.

Не ошибитесь с условием, чтобы случайно не изменить лишние строки!

7 УДАЛЕНИЕ ДАННЫХ ИЗ ТАБЛИЦ

Внезапно, но данные из таблиц иногда нужно удалять.

Можно удалить 1 или несколько записей сразу, а можно очистить таблицу полностью.

Для первого случая необходимо воспользоваться DELETE-запросами.

Ссылка на документацию:
https://postgrespro.ru/docs/postgresql/15/sql-delete

Удалим второй товар из таблицы для примера. Но прежде всего следует указать, что нельзя удалить данные, от которых зависит что-то ещё. То есть если на какую-то строку в таблице ссылается строка или строки в других таблицах, то эту строку нельзя будет удалить.

```
22  DELETE FROM products WHERE product_id = 2;
23  select * from products;
24
```

Data Output | Explain | Messages | Notifications

	product_id [PK] integer	product_name character varying (100)	product_type_id integer	cost_in numeric (10,2)	cost_out numeric (10,2)	product_count integer	product_photo integer
1	1	Батончик глазированный "...	2	1400.00	1600.00	5	[null]

DELETE FROM - команда удаления данных из таблицы.

products - имя таблицы, из которой удаляются данные.

WHERE - оператор, после которого указываем условия удаления данных. Осторожнее, ведь можно очистить больше, чем надо.

product_id = 2 - условие удаления строки.

Очистить всю таблицу можно двумя способами.

```
28  DELETE FROM products;
29
30  TRUNCATE TABLE products;
31
```

Data Output | Explain | Messages | Notifications

```
TRUNCATE TABLE
```

Либо написать DELETE-запрос без условий, либо написать TRUNCATE-запрос.